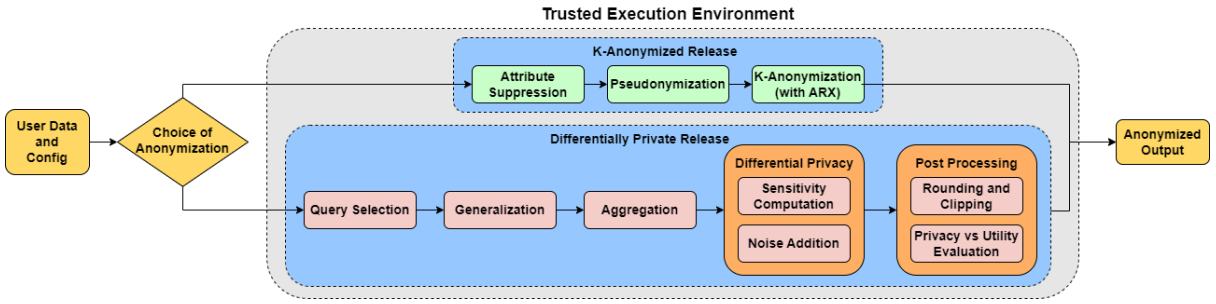
SPIDEr: Validation Document

SPIDEr - **S**ecure **P**ipeline for **I**nformation **D**e-Identification with End-to-End **Enc**r**y**ption, is our implementation of an end-to-end encrypted data de-identification pipeline. SPIDEr supports classical anonymisation techniques such as suppression, pseudonymisation, generalisation, and aggregation, as well as techniques that offer a formal privacy guarantee such as k-anonymisation and differential privacy. To enable scalability and improve performance on constrained TEE hardware, we enable batch processing of data for differential privacy computations.

This document details the control flows for secure execution of de-identification operations within a TEE, validation of the outputs with a given set of inputs, as well as details of attestation, a process that verifies that the software binaries were properly instantiated on a known, trusted platform. A link to the application can be found here: https://dp.ppdx.iudx.org.in/.

Logical Sequence of Operations



This diagram details the logical sequence of operations that is followed by a user when executing the application inside a TEE. The application offers two methods for generating an anonymized output from user data: **K-Anonymization** and **Differential Privacy**. Both processes take user data and a config file as input. A user's choice of anonymization method determines which path is followed.

K-Anonymization

This approach, shown in the top blue box, focuses on ensuring that each record in a dataset is indistinguishable from at least ***k-1*** other records. The user has three options:

1. **Attribute Suppression**: Removing or hiding certain attributes (columns) from the dataset.
2. **Pseudonymization**: Replacing identifying information with pseudonyms.
3. **K-Anonymization (with ARX)**: Applying the k-anonymity principle with an open-source tool known as **ARX**, to group records so that each group has at least *k* members, making it difficult to link a record to a specific individual.

Differential Privacy

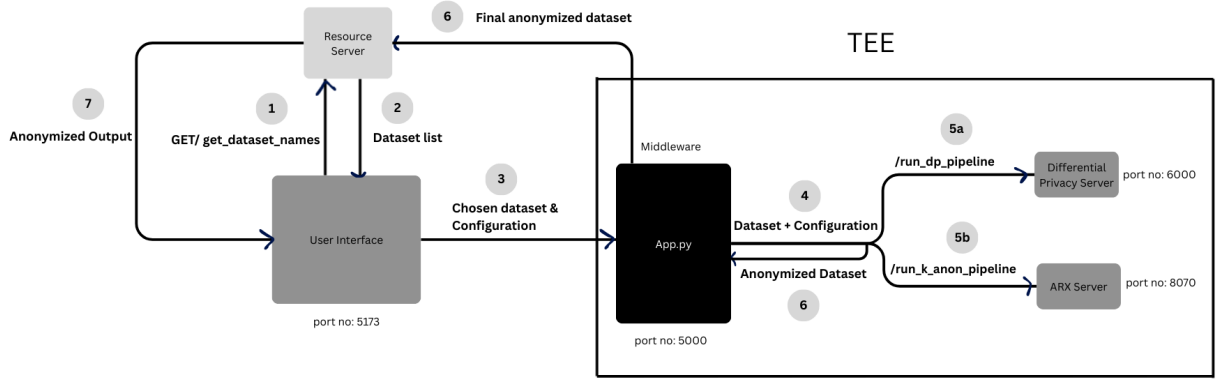
This alternative method, shown in the bottom blue box, aims to add a controlled amount of noise to a dataset to protect individual privacy while still allowing for useful data analysis:

1. **Query Selection**: Choosing the specific queries to be run on the data.
2. **Generalization**: Making data less specific by replacing values with broader categories.
3. **Aggregation**: Grouping data points and computing summary statistics.
4. **Differential Privacy**: This crucial step involves two sub-processes:
    - **Sensitivity Computation**: Determining how much a single individual's data can affect the query's output.
    - **Noise Addition**: Adding a carefully calculated amount of random noise to the query results based on the sensitivity. This ensures that the presence or absence of any single person's data doesn't significantly change the output.
5. **Post-Processing**: The final stage includes:
    - **Rounding and Clipping**: Adjusting the noisy results to make them more useful (e.g., ensuring they are non-negative or within a reasonable range).
    - **Privacy vs Utility Evaluation**: Analyzing the trade-off between the level of privacy achieved and the usefulness (utility) of the anonymized data.

Both of these workflows result in achieving an anonymised output.

Application Workflow Overview

The anonymization workflow is composed of three main layers: User Interface (UI), Middleware (App.py), and pipeline servers responsible for different anonymization strategies (Differential Privacy, and ARX). Below is an outline of the workflow sequence:



Anonymization Flow: Step-by-Step Explanation

1. UI requests dataset names : The User Interface sends a request to the Resource Server to get a list of available datasets.
   request: *GET /get_dataset_names*

2. Resource Server returns dataset list: The Resource Server responds with available datasets .
   Example response: *{ "datasets": ["Medical_Data_new.csv",suratITMS_data.csv"] }*

3. User chooses dataset & configuration: The user selects a dataset and sets anonymization parameters. The UI sends this information to the middleware (*App.py*) inside the TEE.
   Example payload:

Summary


```
{
  "operations": [
    "suppress",
    "pseudonymize",
    "k_anonymize"
  ],
  "dataset_name": "Medical_Data_new.csv",
  "data_type": "medical",
  "medical": {
    "insensitive_columns": [
      "S.No",
      "Age",
      "Gender",
      "PIN Code",
      "Test Result",
      "Blood Pressure"
    ],
    "suppress": [
      "Date of Birth",
      "Address"
    ],
    "pseudonymize": [
      "Name",
      "Patient ID"
    ],
    "generalize": [
      "Height",
      "Weight"
    ],
    "width": {
      "Height": 2,
      "Weight": 3
    },
    "levels": {
      "Height": 10,
      "Weight": 20
    },
    "k_anonymize": {
      "k": 100
    },
    "allow_record_suppression": true
  }
}
```


4. Middleware prepares dataset & config: *App.py* validates the configuration, fetches the raw dataset from the Resource Server into the TEE, and decides which pipeline(s) to run based on the configuration.

5. Pipelines are triggered
   5a — Differential Privacy pipeline:
   App.py calls the Differential Privacy service inside the TEE.
   POST http://127.0.0.1:6000/run_dp_pipeline

   5b — K-Anonymity pipeline:
   App.py calls the K-Anonymity service (ARX-like) inside the TEE.
   POST http://127.0.0.1:8070/run_k_anon_pipeline

6. Pipelines return anonymized dataset: The Differential Privacy and/or K-Anonymity services return their results to *App.py*. *App.py* selects the final anonymized output within the TEE and securely sends the anonymized dataset back to the Resource Server

7. Final anonymized dataset delivered: The UI can then fetch and display or allow download of the final output.

Github Repositories

The various repositories that work in tandem for the execution of SPIDEr are detailed in the table below.

Repository Name	Purpose	Main Files	Ports
iudx-dp-pipeline-ui-v2	UI for uploading datasets and selecting anonymization. Routes to pipelines.	App.tsx,App.py	5173 (UI), 5000 (API)
arx-anonymization	Performs k-anonymization via ARX API.	ARXTestingService.java, ARXTestingController.java	8070
differential-privacy	Applies differential privacy during anonymization.	iudx_dp_server.py, iudx_dp_main.py	6000
resource-server	Stores uploaded datasets and outputs	_init_.py	RS link

Cross-Repository Call Map

The iudx-dp-pipeline-ui-v2 repository serves as the central caller for three distinct functions. First, it uses an HTTP GET request to the Resource Server to fetch a list of available datasets for a dropdown menu, a task handled by the _init_.py file and list_files() function. It then makes an HTTP POST request to the differential-privacy repository to execute the differential privacy code, which is handled by the iudx_dp_server.py file and process_dp() function. Finally, the UI sends another HTTP POST request to the arx-anonymization repository to run an ARX JAR file. This last call is managed by the ARXTestingController.java file and @RequestMapping("/api/arx") function.

Caller Repository	Caller File & Function	Callee Repository	Callee File and Function	Call	Description
iudx-dp-pipeline-ui-v2	App.py -> get_dataset_names()	Resource Server	_init_.py -> list_files():	HTTP GET	Fetch available dataset names

					for dropdown selection
iudx-dp-pipeline-ui-v2	App.py -> /run_dp_pipeline	differential-privacy	iudx_dp_server.py -> process_dp()	HTTP POST	Run the DP code
iudx-dp-pipeline-ui-v2	App.py -> /run_k_anon_pipeline	arx-anonymization	ARXTestingController.java -> @RequestMapping("/api/arx")	HTTP POST	Run the ARX JAR file

TEE Attestation APIs

Guest Attestation is a mechanism to remotely verify the trustworthiness of an instantiated CVM, including cryptographic verification that HW-rooted SNP protection is enabled on the CVM, to shield the CVM workload from Azure operators and Azure hosts. The remote verification is conducted by the Microsoft Azure Attestation (MAA) service.
If you use MAA, you need to implicitly trust that MAA is correctly validating the SNP report and is free from malicious intentions. While MAA follows industry-standards, it remains closed-source.

The enclave manager is a server that provides communication between the enclave and the APD. Its function is to clone the repo of the application to be run, boot the enclave, then build and run the application inside the enclave and extract the output to be returned. It also displays the current state of the application. The different APIs available for the enclave manager are listed in the table below.

Domain	Method	Endpoint	Params (for POST) / Output (for GET)
https://enclave-manager-amd.iudx.io/enclave/inference	GET	/enclave/inference	JSON output based on application

https://enclave-manager-amd.iudx.io/	POST	/enclave/deploy	{
			“url”: “<compose_raw_url>”,
			“rs_url”: “<resource_server_url”,
			“dataset_name” : “<dataset_name>”
			"name": "SGX YOLO APP"
			}

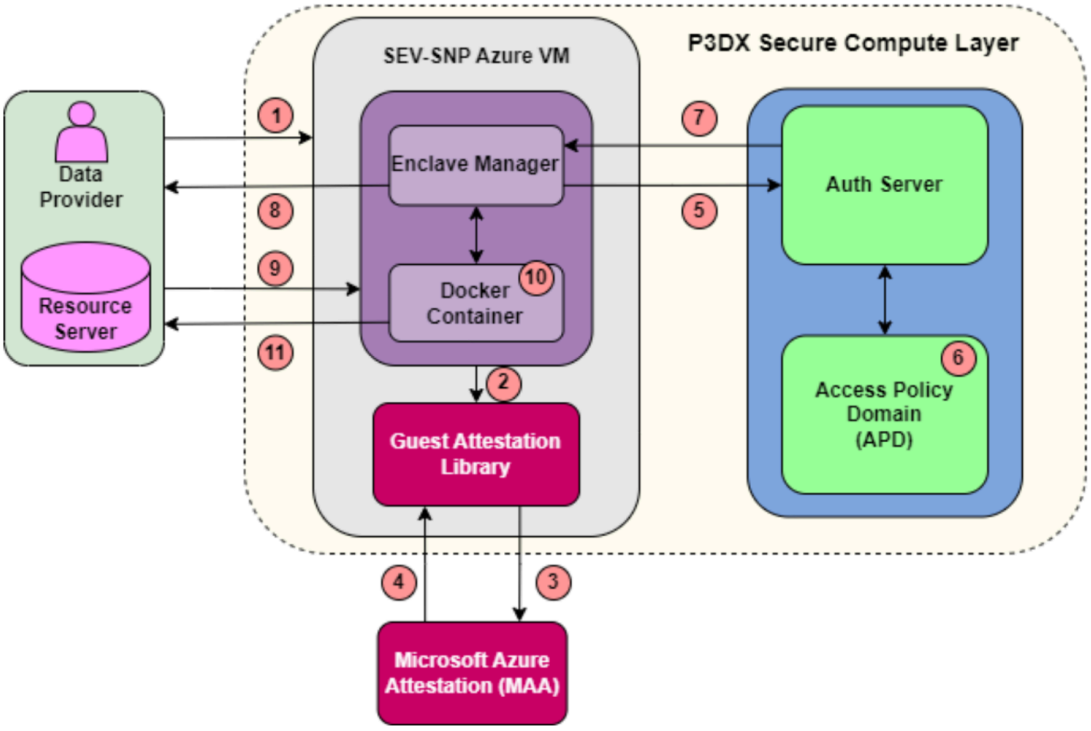
https://enclave-manager-amd.iudx.io/	GET	/enclave/state	{
			"title": "<job title>",
			"description": "<job description>",
			"step": <step no.>,
			"maxSteps":<max no. of steps>
			}

Deploy enclave: The enclave manager brings up the enclave and runs the application inside securely.

Get Inference: The enclave manager returns the output of the application as a JSON object, containing the runOutput and labels.
Get State: The enclave manager returns the current state of the application (title, description and the sequential number of current steps).

TEE Workflow Overview

In order to mitigate the security challenges that accompany hosting a cloud service, we utilise a TEE for confidential computing that can safeguard the code and data during runtime. The hardware-level mechanisms provided by AMD's Secure Encrypted Virtualization with Secure Nested Paging (SEV-SNP) isolate Virtual Machines (VMs) and encrypt their memory, preventing any form of unauthorised access. As part of the control flow for running SPIDEr within the TEE, we perform attestation - a process that verifies that the software binaries were properly instantiated on a known, trusted platform. This provides confidence to remote parties that the intended software is running on trusted hardware. To do this, we take advantage of Microsoft's Azure Attestation (MAA) service that can perform guest-attestation of AMD SEV-SNP based Confidential Virtual Machines (CVMs). The attestation flow is extensible to other cloud service providers provided that attestation and deployment logic is properly handled.



The table below explains the steps from the diagram above.

Step	Action
1	Data Provider initiates a request to run the Secure De-Identification Application.
2	The Guest Attestation Library is invoked to generate the AMD SEV-SNP hardware report.
3	The attestation request (hardware report) is shared with the Microsoft Azure Attestation (MAA) service by the Guest Attestation Library.

4	A JSON Web Token (JWT) is returned by MAA upon successful verification.
5	The JWT is shared with the Auth Server, and then to the Access Policy Domain (APD).
6	The APD verifies the JWT and approves the generation of a Resource Access Token (RAT).
7	The Auth server generates the RAT and shares it with the Enclave Manager.
8	The Enclave Manager requests data access using the RAT.
9	Encrypted data is shared from the Resource Server to the SEV-SNP Azure VM.
10	The application is executed inside a Docker container and an inference is generated.
11	The encrypted inference is sent to the Resource Server.

Proof of Attestation

The MAA service validates the evidence against a user-defined or default policy. This policy is a set of rules that determines what constitutes a "trusted" environment. For example, a policy might check:

- That the enclave is running on genuine, untampered hardware.
- That the hash of the application code running in the enclave matches a known good value.
- That the enclave is not in "debug" mode.

The Claims (The JWT Body):

If the validation is successful, the MAA service populates the JWT with a series of "claims." These claims are the verified facts about the attested environment.

The JWT attests that we are communicating with a genuine Azure Confidential VM leveraging AMD SEV-SNP for hardware-based isolation, that it has booted securely with debug features disabled, and its boot process has been cryptographically measured, that the application running inside the VM has also provided its own public key as part of the attestation. The JWT is signed, self-contained, and tamper-proof.

A sample decoded JWT from a real execution of the pipeline is shared below, along with a breakdown of the key attestation claims.


```
{
  "exp": 1758296814,
  "iat": 1758268014,
  "iss": "https://sharedeus2.eus2.attest.azure.net",
  "jti": "853a91125b0ecc7fc97902946dd3834b460d8d6bb733626c83bf0c5ab10fd995",
  "nbf": 1758268014,
  "secureboot": true,
  "x-ms-attestation-type": "azurevm",
```

```
  "x-ms-azurevm-attestation-protocol-ver": "2.0",
  "x-ms-azurevm-attested-pcrs": [
    0,
    1,
    2,
    3,
    4,
    5,
    6,
    7
  ],
  "x-ms-azurevm-bootdebug-enabled": false,
  "x-ms-azurevm-dbvalidated": true,
  "x-ms-azurevm-dbxvalidated": true,
  "x-ms-azurevm-debuggersdisabled": true,
  "x-ms-azurevm-default-securebootkeysvalidated": true,
  "x-ms-azurevm-elam-enabled": false,
  "x-ms-azurevm-flightsigning-enabled": false,
  "x-ms-azurevm-hvci-policy": 0,
  "x-ms-azurevm-hypervisordebug-enabled": false,
  "x-ms-azurevm-is-windows": false,
  "x-ms-azurevm-kerneldebug-enabled": false,
  "x-ms-azurevm-osbuild": "NotApplication",
  "x-ms-azurevm-osdistro": "Ubuntu",
  "x-ms-azurevm-ostype": "Linux",
  "x-ms-azurevm-osversion-major": 22,
  "x-ms-azurevm-osversion-minor": 4,
  "x-ms-azurevm-signingdisabled": true,
  "x-ms-azurevm-testsigning-enabled": false,
  "x-ms-azurevm-vmid": "DB5AAE36-AA94-43E4-BB97-7AC66937EE37",
  "x-ms-isolation-tee": {
    "x-ms-attestation-type": "sevsnpvm",
    "x-ms-compliance-status": "azure-compliant-cvm",
    "x-ms-runtime": {
      "keys": [
        {
          "e": "AQAB",
          "key_ops": [
            "sign"
          ],
          "kid": "HCLAkPub",
          "kty": "RSA",
          "n":
"oOAgiwAAwqQ6FqrXCiJGfAwWFQyK8TYbNHPp_VC_VjR8M6lCna10NO05gv3DdsQgSxZxM-zmE8l
owL401Ct6YjVYb6QZVX2V8nUsSfSQAjRrQy7DJppnRF_NCq0oPFB89L3v8MuzraOcjjfQTbGJKEP
REvrxCYzmWSIi6w94OVb0rErs37qGgWGSJF9i_seRt9Xlafs_asTThhhAaibZe2QLUQ2pBVkDR-l
```

```
yZ2Ub4B5sEjg5PEjXx2Vu8vBXL8GdUG-F8S6WlU3YHLBw6_wl5Ro0SLwc1GdiZhXzGoI4mybzwcW
M5PGLAIg_yHsFBfkWfsmkhIFFpFGG41FuHZaI8w"
        },
        {
          "e": "AQAB",
          "key_ops": [
            "encrypt"
          ],
          "kid": "HCLEkPub",
          "kty": "RSA",
          "n":
"kFMBMgAA4g-S4FjDQkUaVWU5FAg8SuTDE_mBEGQ8XYiUjCm5OfP780Igo8W1NLJ8rSQHbqz2EtQ
KDVSar6ju41u3o7BEV3YGjxSG3gbJ50CYMHC7-hVnBXLc33VnuaAMZj-X2k85Oo5yZuNI9cwKNvg
_FG42I0jLhNBHwlg1VcjNLn_hn_DNkQGEJqp1rw29tiPiiEFtoXBrdq98mu-V4om_ngIsnLHX8BA
FXs3EDfqUUh7-ib-RVQYkpfFJFBaNytdwzkqox0qqMe4NSlzllwVRaRGWQQNpKCJTrcaaUo8SOMN
W0lUi_hDj6JqnB2lqvkf9WxWXNDgiILuMxoUx2Q"
        }
      ],
      "user-data":
"0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000",
      "vm-configuration": {
        "console-enabled": true,
        "secure-boot": true,
        "tpm-enabled": true,
        "vmUniqueId": "DB5AAE36-AA94-43E4-BB97-7AC66937EE37"
      }
    },
    "x-ms-sevsnpvm-authorkeydigest":
"0000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000",
    "x-ms-sevsnpvm-bootloader-svn": 4,
    "x-ms-sevsnpvm-familyId": "01000000000000000000000000000000",
    "x-ms-sevsnpvm-guestsvn": 8,
    "x-ms-sevsnpvm-hostdata":
"0000000000000000000000000000000000000000000000000000000000000000",
    "x-ms-sevsnpvm-idkeydigest":
"0356215882a825279a85b300b0b742931d113bf7e32dde2e50ffde7ec743ca491ecdd7f336d
c28a6e0b2bb57af7a44a3",
    "x-ms-sevsnpvm-imageId": "02000000000000000000000000000000",
    "x-ms-sevsnpvm-is-debuggable": false,
    "x-ms-sevsnpvm-launchmeasurement":
"ffd92c5d5207afadf3b93be300060a98f9b96bd2a1300c97f1042f2b5f313b964ffc3c14645
a7b706c5f6fe5ccfa51d7",
    "x-ms-sevsnpvm-microcode-svn": 219,
    "x-ms-sevsnpvm-migration-allowed": false,
```

```
    "x-ms-sevsnpvm-reportdata":
"cc75032b90edca4ae06e96de88578fe9624cb65b13cff3ceeef419918c80978100000000000
0000000000000000000000000000000000000000000000000000000000000000",
    "x-ms-sevsnpvm-reportid":
"79f0cbe97bc752428b78d377530736dff2c67da2203cc3dc661c400d2d8c9e96",
    "x-ms-sevsnpvm-smt-allowed": true,
    "x-ms-sevsnpvm-snpfw-svn": 24,
    "x-ms-sevsnpvm-tee-svn": 0,
    "x-ms-sevsnpvm-vmpl": 0
  },
  "x-ms-policy-hash": "wm9mHlvTU82e8UqoOy1Yj1FBRSNkfe99-69IYDq9eWs",
  "x-ms-runtime": {
    "client-payload": {
      "public key":
"TUlJQklqQU5CZ2txaGtpRzl3MEJBUUVGQUFPQ0FROEFNSUlCQ2dLQ0FRRUF1eXdwWGh4YTYyWGZ
uK0lxQ3hCVlplaHRJWlZwNllZZXlTdmdDQndybmV0QjRXVEpFVFZhUWo2M3RJMlZRM2ZVSFY5WTJ
JL2NPQU5ZM1BuYTQrazdnYTZQOFpvOXU5NXppUE9qc1BQKzdEZWhFLzZkbkZQVTZtZkFIOUpxU3Z
rTEtxNXVndGlBVXpsaDJFNStQZlRWSEI0YnpaL0JwaW1DTmlvRDRJMkRiQkk0WmphN2EwVlBoR2g
zWFR1SVhGa1lRZmlKYkVDWDNDNlU5NWJ0YlBIb3JlWThCamNpNDRScVNpWFhaWEZoR2MzNUpsSVd
xc3pEaStWMXd5SlczcGJKc3JDUXBXT2pONnIyWXd4M2IxOUw3d0RRb05wN2RpS2VJWUNQSWVLR0R
Wc2RXZ3BKOTFqWW9UUWMvbkpnY1VlRW9xbHExakxqMjlsR0ZYT1FoZDR1QVFJREFRQUI=",
      "tpm_values":
"eyIwIjogIjB4RTE1QzQ0Nzk2QkVBQkY0NkFCQ0VDN0M1N0U1OTA5NDIwNDFFNDc0OTdFNEVDMjc
1NzFDOEI3NjY0RjQ4RENFRCIsICIxIjogIjB4QUEzMDhERTMxRTY4NUFGOTczNjNCOTU0NkM2NjQ
1Q0NFNUE2RDA1NzM0NDkzNEJFNkZBMDUwMUZGRDY2NDY0RSIsICIyIjogIjB4M0Q0NThDRkU1NUN
DMDNFQTFGNDQzRjE1NjJCRUVDOERGNTFDNzVFMTRBOUZDRjlBNzIzNEExM0YxOThFNzk2OSIsICI
zIjogIjB4M0Q0NThDRkU1NUNDMDNFQTFGNDQzRjE1NjJCRUVDOERGNTFDNzVFMTRBOUZDRjlBNzI
zNEExM0YxOThFNzk2OSIsICI0IjogIjB4NDNBNzYwNzY5RURFNEM0RkMxOTg5QzJFMDI0RkJEMjF
GMkI2RTFBOUVBNzdENEE2NThCQzdGQjdENDc0RjFBNCIsICI1IjogIjB4OTJDMjgyN0M2NkQxRTc
0MzYyRjVFOEI5QzVENzg0MkEwQzVFQ0VFNjc1OTg0QzVEM0JFQTNCQkJDOTk0REJGMiIsICI2Ijo
gIjB4NEJFNzgxQUFEMEU3RTcwMTQ1QTIyMTlBQ0IwQ0M2QjFCNDUyNDkwNEYxOUYwRjFFMDE1QzF
EMjgzQ0RGODhDNyIsICI3IjogIjB4MzJGRTQyQjM4NUI0N0NCMjJDOTA2QjhBN0U0RjEzNEU5RjI
yNzA4MThGOTBFOTQwNzJEMTEwMUVGNzJGMUMwMCIsICI4IjogIjB4MDAwMDAwMDAwMDAwMDAwMDA
wMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMDAwMCIsICIxNSI6ICI
weDhDREM0Qzc0ODIxMjg1RDE4RjNCNDRDOTU3NTAyRjE5MjA4OEQ0QjgyMTdDOEFDQjk0ODE2REI
3NjNBNzVFMzgifQ=="
    },
    "keys": [
      {
        "e": "AQAB",
        "key_ops": [
          "encrypt"
        ],
        "kid": "TpmEphemeralEncryptionKey",
        "kty": "RSA",
        "n":
```

```
"kBJKtQAAsT0IWhXXvbAZJRohYTDkZuHasJgqN_u7LLA0vXRCuG3V1o7RnGFDF_6QTOv3FpWPxcl
KErr50VwPiwhfzPp25k3C3eUALMmYBeO7UCs8__lxgPaLYIJcpu7R-KhjFcqDJCSUrfiQIY-t06Y
F8_-f1I_i5W4XjwxTQBJdm1Z5vOCGvsboX1llzHanna5v9MqBAGJ2y1cTxzVNPfTL6IR1n73O1tg
Qz_IEsVazMG2N2WE3rseLoetBcKnrqyc7gqtyJD62mZumdN4KkMSf0v4x7bqbidnNSX2EwCd942Z
V14Qe_R7JkoIpkIncIrxw5EWDFuK9bjyLv0fxZQ"
      }
    ]
  },
  "x-ms-ver": "1.0"
}
```


The key takeaways from the JWT are:

General VM and Security Posture

`"secureboot": true`: Confirms the VM booted using UEFI Secure Boot, meaning only trusted and signed bootloaders and kernels were loaded.

`"x-ms-attestation-type": "azurevm"`: Identifies the attested resource as a standard Azure VM.

`"x-ms-azurevm-ostype": "Linux"`: The operating system is Linux.

`"x-ms-azurevm-osdistro": "Ubuntu"` & `"x-ms-azurevm-osversion-major": 22`: Confirms the OS is Ubuntu 22.

`"x-ms-azurevm-debuggersdisabled": true`, `"x-ms-azurevm-kerneldebug-enabled": false`: These claims confirm that various debugging interfaces are disabled, which is the expected and secure state for a production environment.

`"x-ms-azurevm-vmid": "DB5AAE36-..."`: The unique identifier for this specific Azure VM instance.

Confidential Computing (SEV-SNP) Evidence

The nested object under `"x-ms-isolation-tee"` contains the hardware-level proof from the AMD processor.

`"x-ms-attestation-type": "sevsnpvm"`: This is the key indicator that the VM is a confidential VM using AMD SEV-SNP.

`"x-ms-compliance-status": "azure-compliant-cvm"`: This is Microsoft's assertion that the underlying host and guest configuration meet the security requirements for an Azure Confidential VM.

`"x-ms-sevsnpvm-launchmeasurement": "ffd92c5d..."`: This is the most important piece of evidence. It is a cryptographic hash (a "measurement") of the entire initial state of the VM, including the kernel and initial RAM disk. To verify the integrity of the workload, this value is compared against a known, trusted measurement for the specific application image.

`"x-ms-sevsnpvm-reportdata": "cc75032b..."`: This is 64 bytes of custom data that the client application inside the VM provided when it requested the attestation. It is used to prevent replay attacks by linking this specific hardware report to the current transaction.

`"x-ms-sevsnpvm-is-debuggable": false`: Confirms from the hardware level that the VM is not in a debuggable state.

`"vm-configuration": { "secure-boot": true, "tpm-enabled": true }`: The hardware report itself confirms that Secure Boot and a virtual Trusted Platform Module (vTPM) are enabled for the guest VM.

Client-Provided Data

The `"x-ms-runtime"` object contains data injected by the application running inside the CVM.

`"client-payload"`: Data provided by the application.

`"public key": "TUlJQklqQU5..."`: The application has presented its own Base64-encoded public key. Because this key is included in a cryptographically signed attestation report, it can be trusted to belong to the application running inside this specific, verified, and secure VM.

`"tpm_values"`: These are the Platform Configuration Register (PCR) values from the vTPM, which represent the measurements of the boot components. These can be compared against known "golden" values for the specific OS and boot chain.

Application I/O Validation

Differential Privacy

Differential Privacy (DP) is a mathematical framework that ensures statistical queries on a dataset do not reveal sensitive information about any single individual. Noise is added to the computed query results, calibrated to a privacy budget (ε).

For this dataset, DP is applied to compute the mean speed of vehicles grouped by '*HAT*', '*Date*', and '*license_plate*'. By injecting noise into the mean, the contribution of any single license plate is hidden, thus preserving privacy while still allowing aggregate analysis.

Input Data (Snippet)

Below are the first five rows of the input dataset before anonymization.

actual_trip_start_time	id	last_stop_arrival_time	last_stop_id	license_plate	location.coordinates	location.type	observationDateTime	route_id	speed	trip_delay	trip_direction	trip_id	vehicle_label
2022-11-01T05:18:26+05:30	0		10	GJ05BZ0884	[72.840186, 21.204533]	Point	2022-11-01 05:29:26+05:30	137JU	0	47	UP	20180951	H159
2022-11-01T05:14:04+05:30	1	04:54:03	10	GJ05BX3044	[72.861378, 21.158777]	Point	2022-11-01 05:29:35+05:30	104U	0	320	UP	20179843	M05
2022-11-01T05:18:26+05:30	2		10	GJ05BZ0884	[72.840186, 21.204537]	Point	2022-11-01 05:29:38+05:30	137JU	0	47	UP	20180951	H159
2022-11-01T05:05:52+05:30	3		4449	GJ05BX3043	[72.88417, 21.14024]	Point	2022-11-01 05:30:04+05:30	104D	0	77	DN	20179800	M04
2022-11-01T05:18:26+05:30	4		10	GJ05BZ0884	[72.840186, 21.204537]	Point	2022-11-01 05:30:06+05:30	137JU	0	47	UP	20180951	H159

Configuration file

The following configuration file is automatically generated when the user fills out the DP form in the UI:

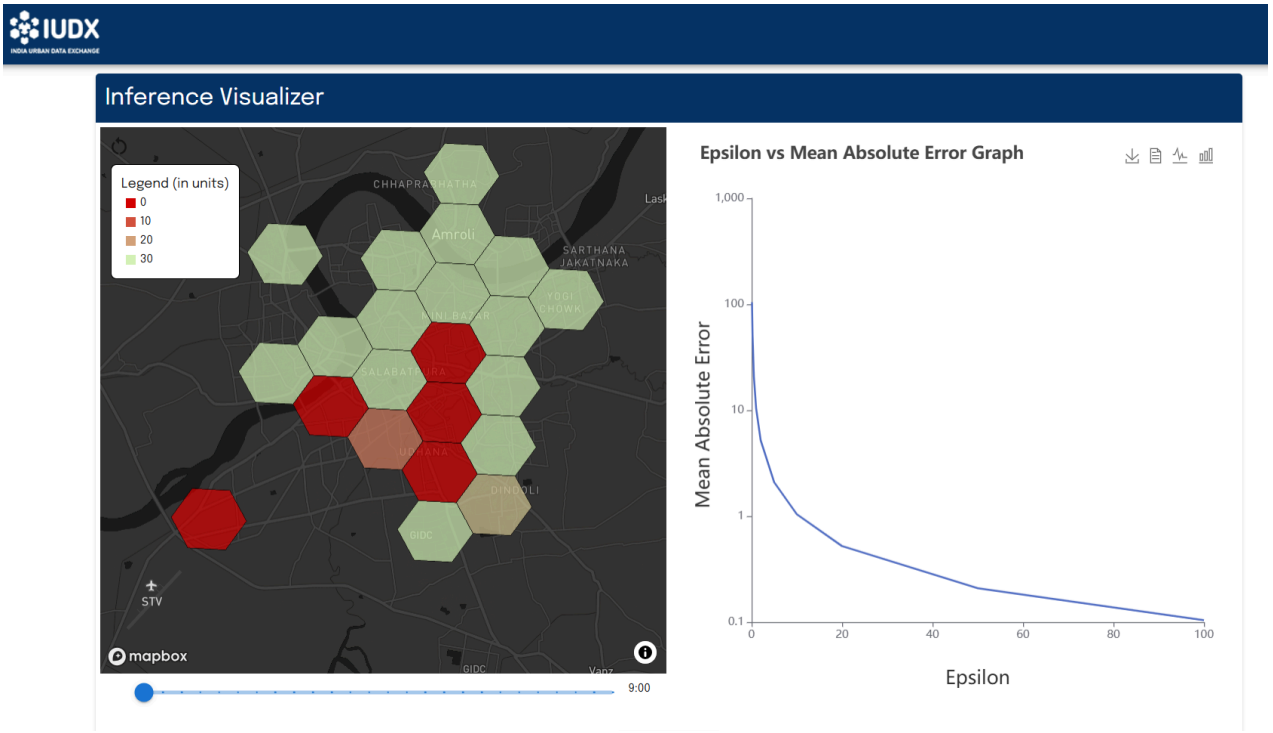
```
{
  "operations": ["dp"],
  "dataset_name": "suratITMS_data.csv",
  "data_type": "spatioTemporal",
  "spatioTemporal": {
    "spatial_generalize": {
      "spatial_attribute": "location.coordinates",
      "filter_attribute": "license_plate",
      "filter_attribute_by": ["HAT", "Date"],
      "h3_resolution": 7,
      "filter_event_occurences": 7
    },
    "temporal_generalize": {
      "temporal_attribute": "observationDateTime",
      "timeslot_resolution": 30,
      "start_time": 9,
      "end_time": 22
    },
    "differential_privacy": {
      "dp_aggregate_attribute": ["HAT", "Date", "license_plate"],
      "global_max_value": 65,
      "global_min_value": 0,
      "dp_epsilon_step": 0.1,
      "dp_output_attribute": "speed",
      "dp_query": "mean",
      "dp_epsilon": 0.1
    }
  }
}
```

Configuration Parameter	Description
dataset_name	Input dataset file name

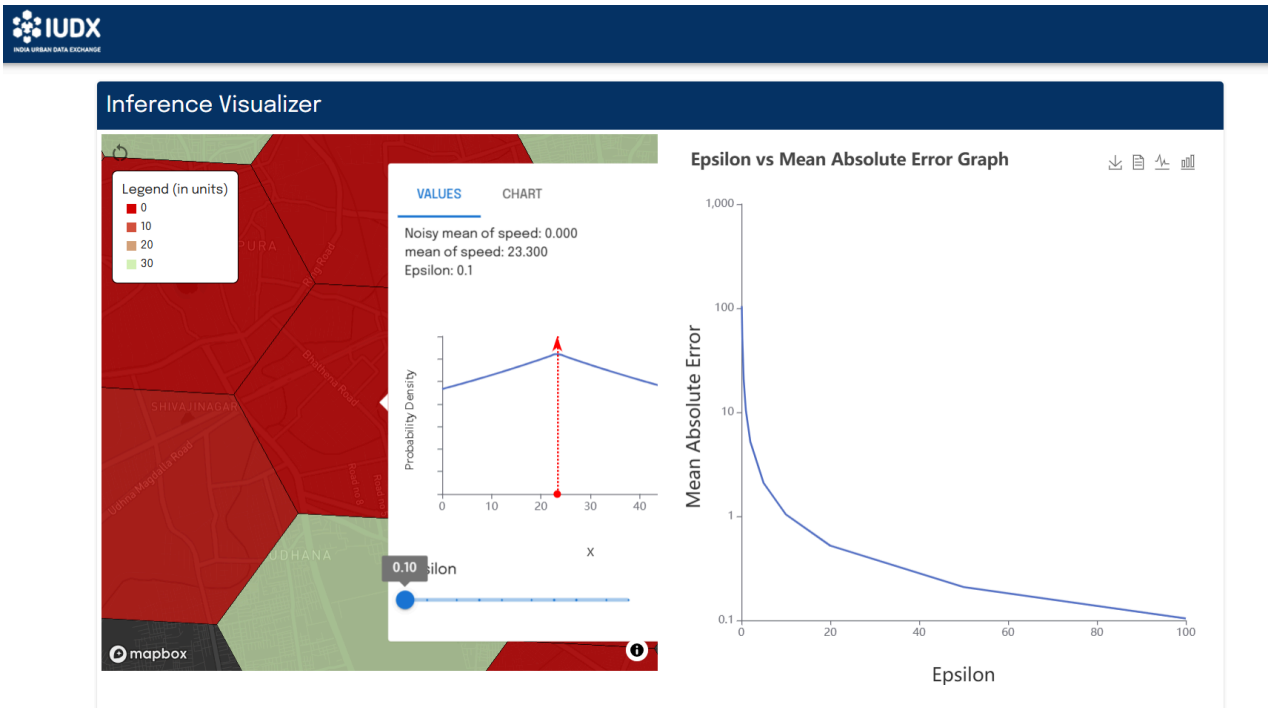
data_type	The type of data; here spatioTemporal
spatial_attribute	Column holding coordinates
filter_attribute	Attribute used to filter unique entities (e.g., license plate)
h3_resolution	Resolution level for spatial granularity
filter_event_occurrences	Minimum occurrences for inclusion
temporal_attribute	Column holding timestamps
timeslot_resolution	Time interval in minutes
start_time / end_time	Time window of interest
dp_aggregate_attribute	Grouping columns for aggregation
dp_output_attribute	Attribute on which DP query is applied (e.g., speed)
dp_query	Statistical operation (e.g., mean)
global_min_value / global_max_value	Bounds for clipping data
dp_epsilon	Privacy budget value
dp_epsilon_step	Step size for experimentation across multiple epsilons

Results and Validation

The system produces visualizations to showcase the output of the application.



The Epsilon vs. Mean Absolute Error plot is generated to show the trade-off between privacy and utility. Increasing Epsilon results in reduced Mean Absolute Error, meaning that an increase in privacy directly results in a decrease in utility.



By selecting individual HATs on the map, the UI displays the true counts alongside the noisy counts for different values of ε, as a demonstration of the relationship between noise and epsilon.

These outputs serve as the functional proof that the dataset has been anonymized using differential privacy.

Expected Log

```
    # Step 1 - Pulling docker compose & extracting docker image link
    print("\nStep 1")
    box_out("Pulling Docker Compose from GitHub...")
    PPDX_SDK_DP.pull_compose_file(github_raw_link)
    print('Extracting docker link...')
    link = subprocess.check_output(["sudo", "docker", "compose", "config",
"--images"]).decode().strip()
    print("Image information:", link)

    # Step 2 - Key generation
    print("\nStep 2")
    box_out("Generating and saving key pair...")
    PPDX_SDK_DP.setState("TEE Attestation & Authorisation", "Step
2",2,5,address)
    key=PPDX_SDK_DP.generate_and_save_key_pair()

    # Step 3 - Docker image pulling
    print("\nStep 3")
    box_out("Pulling docker image...")
    PPDX_SDK_DP.pull_docker_image(link)
    print("Pulled docker")

    # Step 4 - Measuring image and storing in vTPM
    print("\nStep 4")
    box_out("Measuring Docker image into vTPM...")
    PPDX_SDK_DP.measureDockervTPM(link)
    print("Measured and stored in vTPM")

    # Step 5 - Send VTPM & public key to MAA & get attestation token
    print("\nStep 5")
    box_out("Guest Attestation Executing...")
    PPDX_SDK_DP.execute_guest_attestation()
    print("Guest Attestation complete. JWT received from MAA: ")

    # Step 6 - Send the JWT to APD
    print("\nStep 6")
    box_out("Sending JWT to APD for verification...")
    token=PPDX_SDK_DP.getTokenFromAPD('jwt-response.txt', config, dataset,
rs_url)
    print("Access token received from APD")

    # Step 7 - Pulling config file from RS:
    print("\nStep 7")
    box_out("Pulling DP application config")
```

```
    PPDX_SDK_DP.setState("Getting data into Secure enclave","Step
3",3,5,address)
    PPDX_SDK_DP.pullconfig(config_url, token, key)
    print("Config pulled & stored")

    # Step 8 - Pulling chunks, decrypting & storing
    print("\nStep 8")
    box_out("Getting files from RS, decrypting and storing locally...")
    count=0
    lengthList=[]
    while True:
        count=count+1
        print("The lengthList is: ", lengthList)

        ret =  PPDX_SDK_DP.dataChunkN(count, data_url, token, key)
        if (ret==0):
            print ("Error retrieving chunk. Assuming no more chunks..")
            break
        lengthList.append(count)

    total_chunks = len(lengthList)
    print("TOTAL chunks stored :", total_chunks)

    # Executing the application in the docker
    print("\nStep 9")
    box_out("Running the Application...")
    PPDX_SDK_DP.setState("Performing secure de-identification in TEE", "Step
4",4,5,address)
    subprocess.run(["sudo", "docker", "compose", 'up'])
    print('Output saved to /tmp/output')

    print("\nStep 10")
    print("Getting inference encryption key from RS..")
    inference_key=PPDX_SDK_DP.getInferenceFernetKey(key, inferencekey_url,
token)
    print("Got back inference key: ", inference_key)

    print("\nStep 11")
    print("Encrypting Inference using inference key")
    inference=PPDX_SDK_DP.encryptInference(inference_key)
    print("Inference encrypted")

    print("\nStep 12")
    print("Sending encrypted inference to RS")
    PPDX_SDK_DP.sendInference(inference, token, inference_url)
    print("Inference sent to RS")
```

```
    print('DONE')
    PPDX_SDK_DP.setState("Secure Computation Complete", "Step 5", 5, 5,
address)
```

K-anonymization

K-Anonymity ensures that each individual record in a dataset is indistinguishable from at least k–1 other records with respect to quasi-identifiers. This protects against identity disclosure by grouping records into equivalence classes of size ≥ k.

For this dataset, K-anonymization is applied to medical data. Identifiers like '*Patient ID'* and '*Name'* are pseudonymized, sensitive details such as '*Date of Birth'* and '*Address'* are suppressed, and quasi-identifiers ('*Height'*, '*Weight'*, '*Age'*, '*PIN Code'*) are generalized until each equivalence class contains at least 100 records.

Input Data (Snippet)

Below are five rows of the dataset before anonymization:

S.No	Address	Age	Blood Pressure	Date of Birth	Gender	Height	Name	PIN Code	Patient ID	Test Result	Weight
2	6182 Ashley Key South	37	177/105	1953-07-20	Male	160.4	Christine Casey	3300938	4be55da4-27ee-4991-a1f8-21a90f1572b1	'-ve	76.7
3	Unit 0227 Box 3028 DP	73	175/72	1965-01-18	Male	149.5	Christopher Thompson	2200675	f3c076f5-737b-40fc-9666-b980a976c4ce	'-ve	64.7
4	64201 Contreras Brook	16	166/100	1983-09-22	Male	125.7	Nicholas Fletcher	2200967	d10c570e-aa7e-438b-be48-ae1b38657cc5	'-ve	31.7
5	USNS Robbins FPO AA	1	159/87	1940-09-12	Male	131.3	David Stanton	2200636	0c1bc076-da56-4660-b331-805822591866	'+ve	57.7
6	464 Howell Brook New	59	179/65	2022-07-12	Female	162.7	Alan Travis	3300564	21242342-2750-49b4-97c3-aab51893106f	'-ve	72.1

Configuration File

Configuration Parameter	Description
insensitive_columns	Columns that can remain as-is without risk of re-identification.
suppress	Columns removed entirely to protect privacy (e.g., Date of Birth, Address).
pseudonymize	Columns replaced with a hashed value (e.g., Patient ID, Name).
generalize	Quasi-identifiers to be generalized (e.g. Height, Weight, Age, PIN Code).
width	Step size for binning each quasi-identifier.
levels	Maximum number of generalization levels for each quasi-identifier.
k	Minimum size of each equivalence class.
allow_record_suppression	Whether records can be suppressed if they cannot meet the K-anonymity requirement.

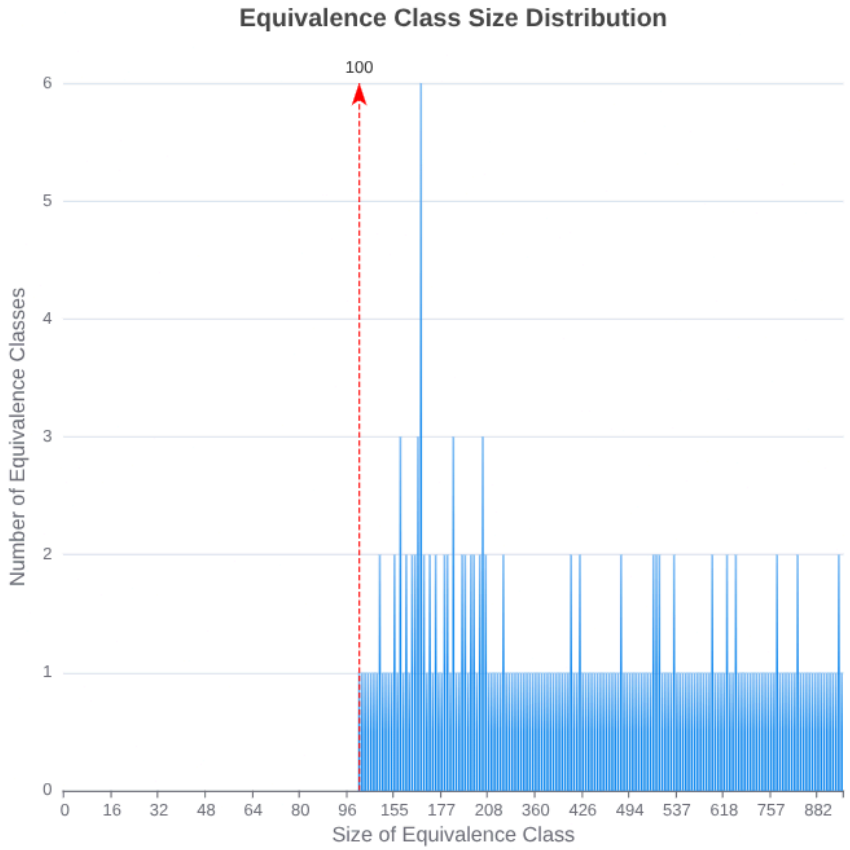
The following configuration file is automatically generated when the user fills out the K-anonymization form in the UI:

```
{
  "operations": ["suppress", "pseudonymize", "k_anonymize"],
  "dataset_name": "Medical_Data_new.csv",
  "data_type": "medical",
  "medical": {
    "insensitive_columns": ["S.No", "Gender", "Test Result", "Blood
Pressure"],
    "suppress": ["Date of Birth", "Address"],
    "pseudonymize": ["Patient ID", "Name"],
    "generalize": ["Height", "Weight", "Age", "PIN Code"],
    "width": {
      "Height": 1,
      "Weight": 1,
      "Age": 1
    },
    "levels": {
      "Height": 10,
      "Weight": 14,
      "Age": 12
    },
    "k_anonymize": {
      "k": 100
    },
    "allow_record_suppression": true
  }
}
```

Results and Validation

S.No	Age	Blood Pressure	Gender	Height	Name	PIN Code	Patient ID	Test Result	Weight
2	[33.0, 49.0]	177/105	Male	[154.0, 170.0]	43729033d4b82	3300***	208db4ce38528	'-ve	[63.0, 79.0]
3	[65.0, 81.0]	175/72	Male	[138.0, 154.0]	bd96fe0f3f1d95b	2200***	ac0355e3a6226	'-ve	[63.0, 79.0]
4	[1.0, 17.0]	166/100	Male	[122.0, 138.0]	ff82a20c50cc60e	2200***	600af9a691174	'-ve	[31.0, 47.0]
5	[1.0, 17.0]	159/87	Male	[122.0, 138.0]	2fdf5f1bb78d904	2200***	e8a59984b26b8	'+ve	[47.0, 63.0]
6	[49.0, 65.0]	179/65	Female	[154.0, 170.0]	64a5b65ee80fb8	3300***	f1b282aa64ae8	'-ve	[63.0, 79.0]

The resulting anonymized dataset contains suppressed, pseudonymized, and generalized values, ensuring that each record is part of an equivalence class of at least $k = 100$.



The primary proof that K-anonymization has been successfully applied is provided by the Equivalence Class Plot.

The plot displays the size distribution of equivalence classes, where each class groups records sharing identical values for the quasi-identifiers. K-anonymity requires that every equivalence class contains at least k records (here, k = 100). The plot confirms that this condition is satisfied across the dataset as the minimum equivalence class size is 100.

This visual validation, together with the anonymized dataset itself, serves as the functional proof that the anonymization process has been correctly executed.

Expected Log

```
import subprocess
import os
import PPDX_SDK_DP
import sys
import json
import shutil
import time
import :psutil
import logging
```

```
from datetime import datetime

def load_config(filename):
    """Loads configuration data from a JSON file."""
    try:
        with open(filename, 'r') as file:
            return json.load(file)
    except FileNotFoundError:
        raise FileNotFoundError(f"Configuration file '{filename}' not
found.")
    except json.JSONDecodeError:
        raise ValueError(f"Invalid JSON format in configuration file
'{filename}'.")

def box_out(message):
    """Prints a box around a message using text characters."""

    lines = message.splitlines()  # Split message into lines
    max_width = max(len(line) for line in lines)  # Find longest line

    # Top border
    print("+" + "-" * (max_width + 2) + "+")

    # Content with padding
    for line in lines:
        print("| " + line.ljust(max_width) + " |")

    # Bottom border
    print("+" + "-" * (max_width + 2) + "+")

def remove_files():
    file_path = os.path.join('.','docker-compose.yml')
    if os.path.exists(file_path):
        os.remove(file_path)
        print(f"Removed file: {file_path}")
    else:
        print(f"File not found: {file_path}")

    folder_path = os.path.join('.', 'keys')
    if os.path.exists(folder_path):
        shutil.rmtree(folder_path)
        print(f"Removed folder and contents: {folder_path}")
    else:
        print(f"Folder not found: {folder_path}")

    folder_path = os.path.join('/tmp', 'arx_input')
```

```
    if os.path.exists(folder_path):
        shutil.rmtree(folder_path)
        print(f"Removed folder and contents: {folder_path}")
    else:
        print(f"Folder not found: {folder_path}")

    os.makedirs(folder_path)
    print("Created input folder")

    config_path = os.path.join(folder_path, "config")
    os.makedirs(config_path)

    encdata_path = os.path.join(folder_path, "encrypted_data")
    os.makedirs(encdata_path)

    inputdata_path = os.path.join(folder_path, "input_file")
    os.makedirs(inputdata_path)

    folder_path = os.path.join('/tmp', 'arx_output')
    if os.path.exists(folder_path):
        shutil.rmtree(folder_path)
        print(f"Removed folder and contents: {folder_path}")
    else:
        print(f"Folder not found: {folder_path}")

    os.makedirs(folder_path)
    print("Created output folder")

# Start the main process
if __name__ == "__main__":
    print("In DP script main")
    if len(sys.argv) < 2:
        print("Error: Missing arguments.")
        sys.exit(1)  # Exit with an error code

    dataset = sys.argv[1]
    rs_url = sys.argv[2]
    github_raw_link = sys.argv[3]

    # Validate the link
    if not github_raw_link.startswith("https://raw.githubusercontent.com/"):
        print("Error: Invalid GitHub raw link format.")
        sys.exit(1)

    remove_files()
```

```
    config_file = "DPconfig.json"
    config = load_config(config_file)  # Loads configuration into a
dictionary
    address = config["enclaveManagerAddress"]

    inference_url = f"{rs_url}/inference/{dataset}"
    inferencekey_url = f"{rs_url}/key/{dataset}"
    data_url = f"{rs_url}/data/{dataset}/"
    config_url = f"{rs_url}/config/{dataset}"

    # Step 1 - Pulling docker compose & extracting docker image link
    print("\nStep 1")
    box_out("Pulling Docker Compose from GitHub...")
    PPDX_SDK_DP.pull_compose_file(github_raw_link)
    print('Extracting docker link...')
    link = subprocess.check_output(["sudo", "docker", "compose", "config",
"--images"]).decode().strip()
    print("Image information:", link)

    # Step 2 - Key generation
    print("\nStep 2")
    box_out("Generating and saving key pair...")
    PPDX_SDK_DP.setState("TEE Attestation & Authorisation", "Step
2",2,5,address)
    key=PPDX_SDK_DP.generate_and_save_key_pair()

    # Step 3 - Docker image pulling
    print("\nStep 3")
    box_out("Pulling docker image...")
    PPDX_SDK_DP.pull_docker_image(link)
    print("Pulled docker")

    # Step 4 - Measuring image and storing in vTPM
    print("\nStep 4")
    box_out("Measuring Docker image into vTPM...")
    PPDX_SDK_DP.measureDockervTPM(link)
    print("Measured and stored in vTPM")

    # Step 5 - Send VTPM & public key to MAA & get attestation token
    print("\nStep 5")
    box_out("Guest Attestation Executing...")
    PPDX_SDK_DP.execute_guest_attestation()
    print("Guest Attestation complete. JWT received from MAA: ")

    # Step 6 - Send the JWT to APD
    print("\nStep 6")
```

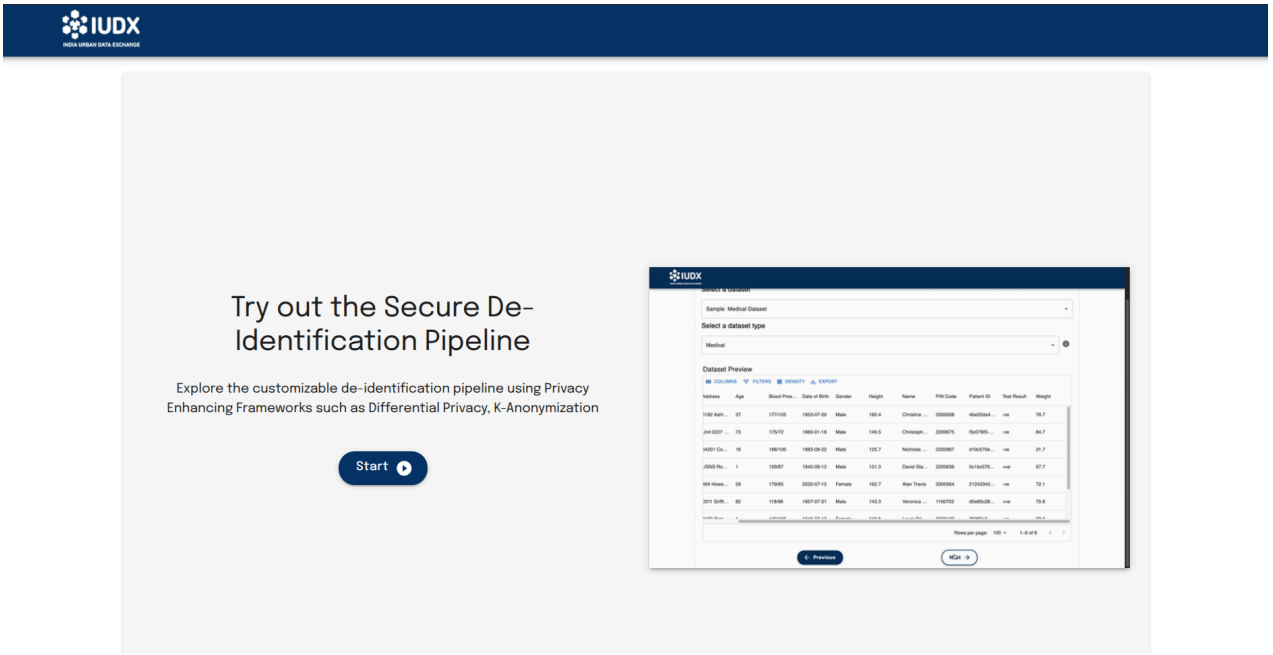
```
    box_out("Sending JWT to APD for verification...")
    token=PPDX_SDK_DP.getTokenFromAPD('jwt-response.txt', config, dataset,
rs_url)
    print("Access token received from APD")

    # Step 7 - Pulling config file from RS:
    print("\nStep 7")
    box_out("Pulling DP application config")
    PPDX_SDK_DP.setState("Getting data into Secure enclave","Step
3",3,5,address)
    PPDX_SDK_DP.pullconfig_KAnon(config_url, token, key)
    print("Config pulled & stored")

    # Step 8 - Pulling chunks, decrypting & storing
    print("\nStep 8")
    box_out("Getting files from RS, decrypting and storing locally...")
    count=0
    lengthList=[]
    while True:
        count=count+1
        print("The lengthList is: ", lengthList)

        ret =  PPDX_SDK_DP.dataChunkN_KAnon(count, data_url, token, key)
        if (ret==0):
            print ("Error retrieving chunk. Assuming no more chunks..")
            break
        lengthList.append(count)

    total_chunks = len(lengthList)
    print("TOTAL chunks stored :", total_chunks)

    # Executing the application in the docker
    print("\nStep 9")
    box_out("Running the Application...")
    PPDX_SDK_DP.setState("Performing secure de-identification in TEE", "Step
4",4,5,address)
    subprocess.run(["sudo", "docker", "compose", 'up'])
    print('Output saved to /tmp/output')

    print("\nStep 10")
    print("Getting inference encryption key from RS..")
    inference_key=PPDX_SDK_DP.getInferenceFernetKey(key, inferencekey_url,
token)
    print("Got back inference key: ", inference_key)

    print("\nStep 11")
```

```
    print("Encrypting Inference using inference key")
    inference=PPDX_SDK_DP.encryptInference_KAnon(inference_key)
    print("Inference encrypted")

    print("\nStep 12")
    print("Sending encrypted inference to RS")
    PPDX_SDK_DP.sendInference(inference, token, inference_url)
    print("Inference sent to RS")

    print('DONE')
    PPDX_SDK_DP.setState("Secure Computation Complete", "Step 5", 5, 5,
address)
```

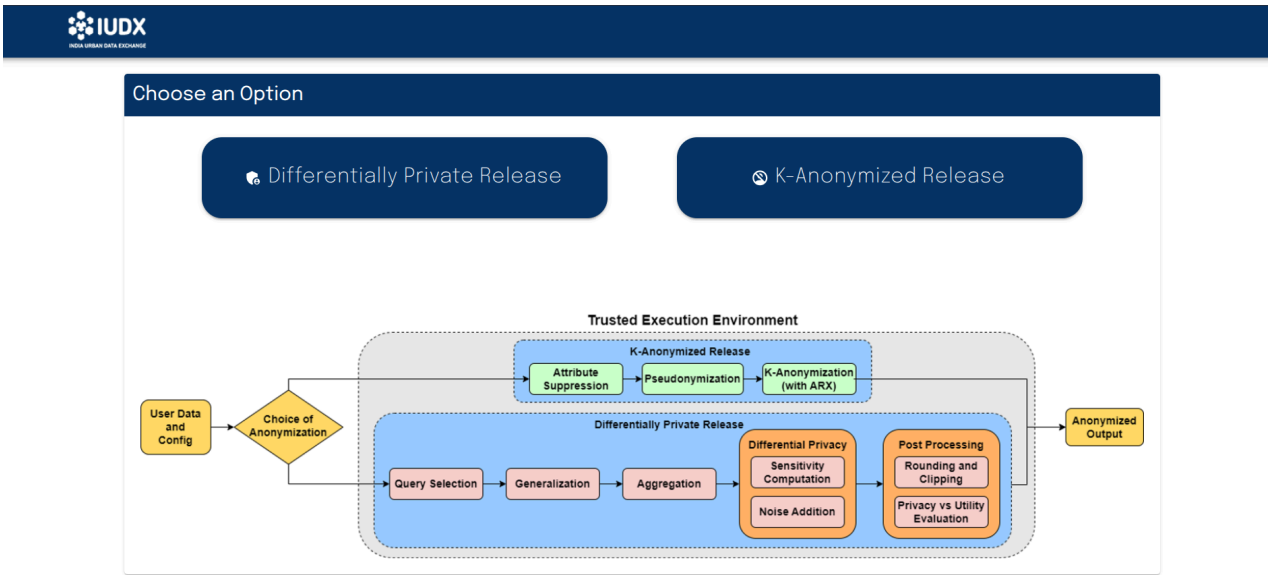
Appendices

Appendix A: User Guide

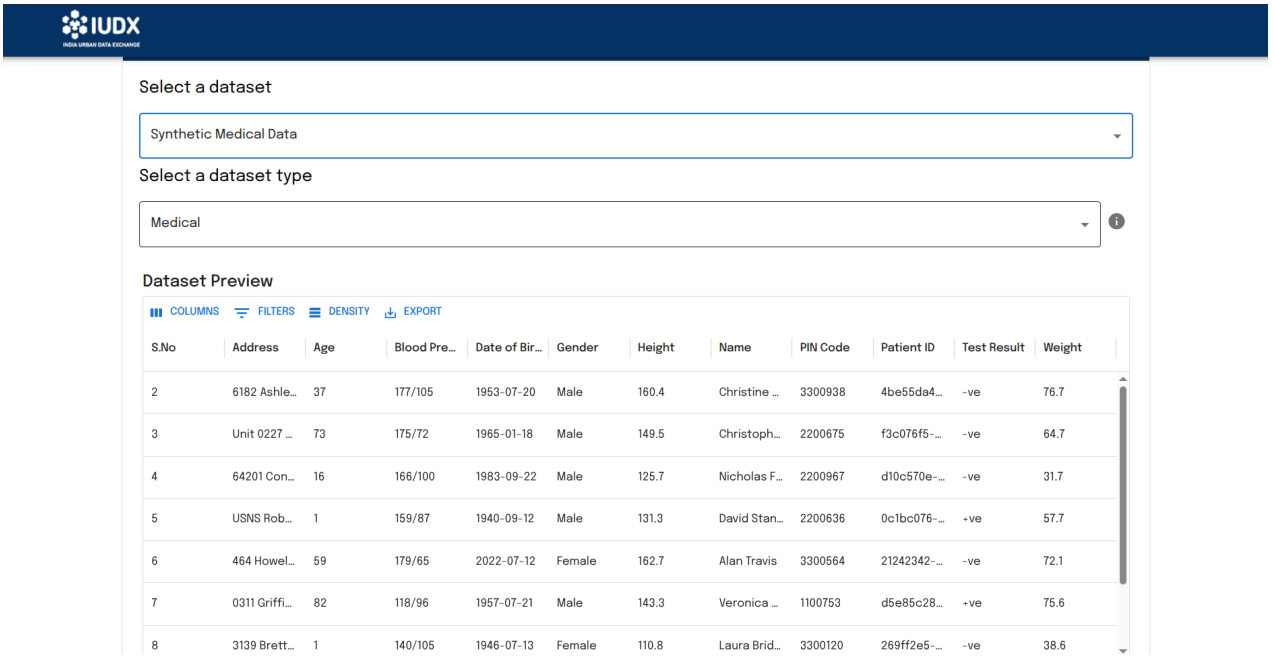
Below is a step by step guide for running the end to end encrypted anonymisation workflow (SPIDEr) via UI. The snapshots for the step-by-step guide are presented below.



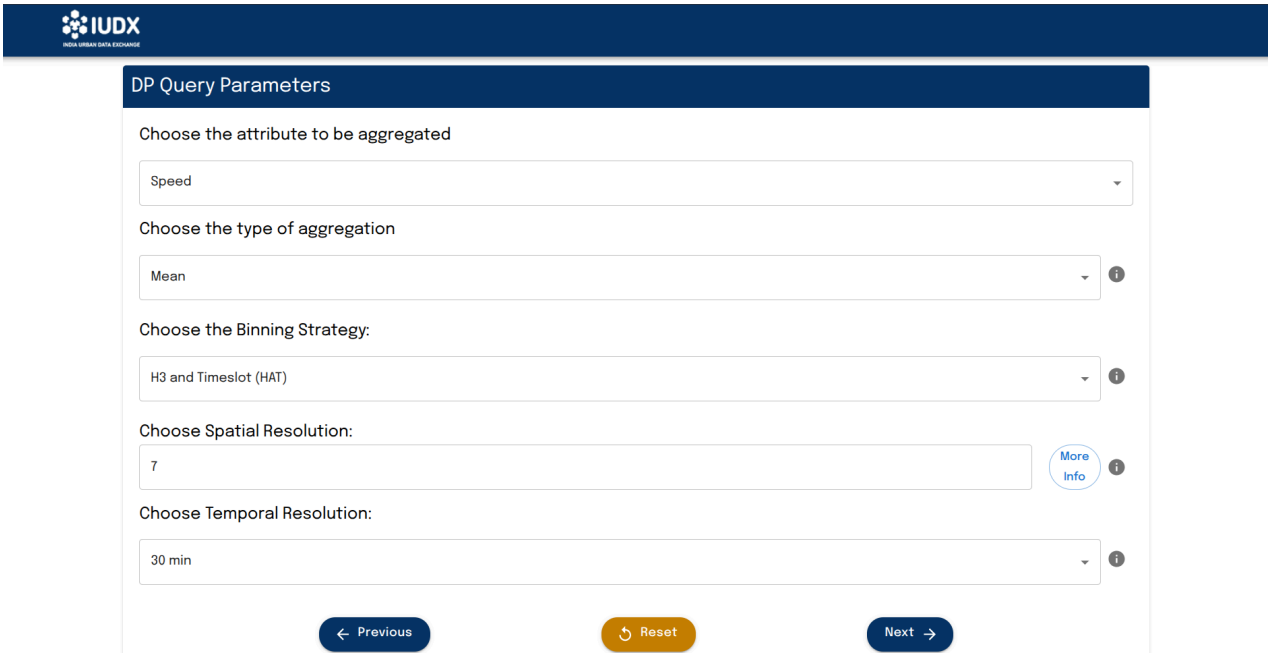
Step 1: Click on *Start* to navigate to the next page.



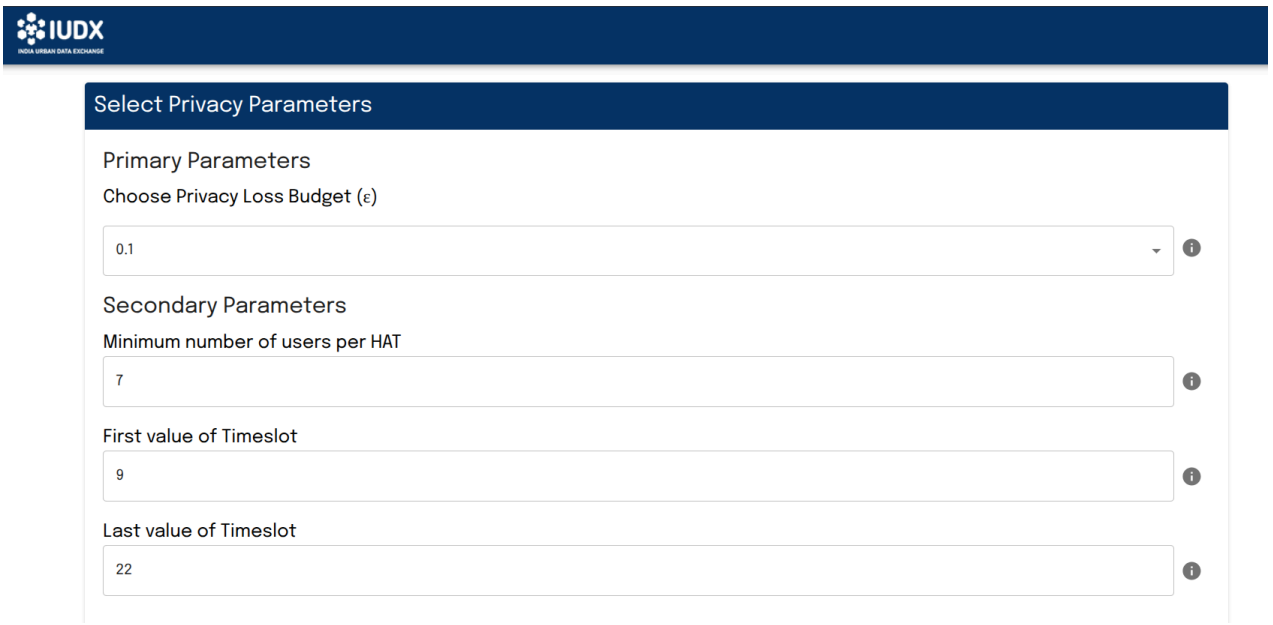
Step 2: Choose either *Differentially Private release* or *K-anonymized release* to proceed.



Step 3a: After clicking on Differentially Private Release, select the dataset and its type from the dropdown menus.



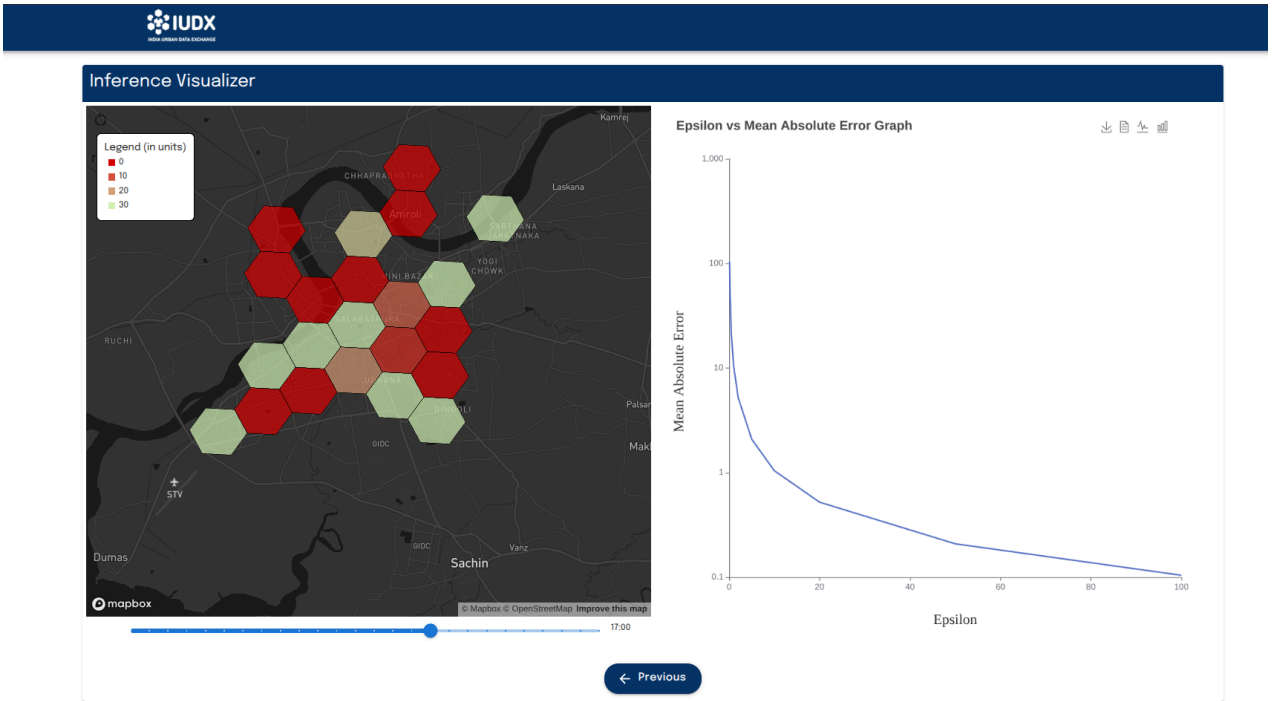
Step 4a: Enter your preferred values for each Differential Privacy query parameter to be applied during anonymization.



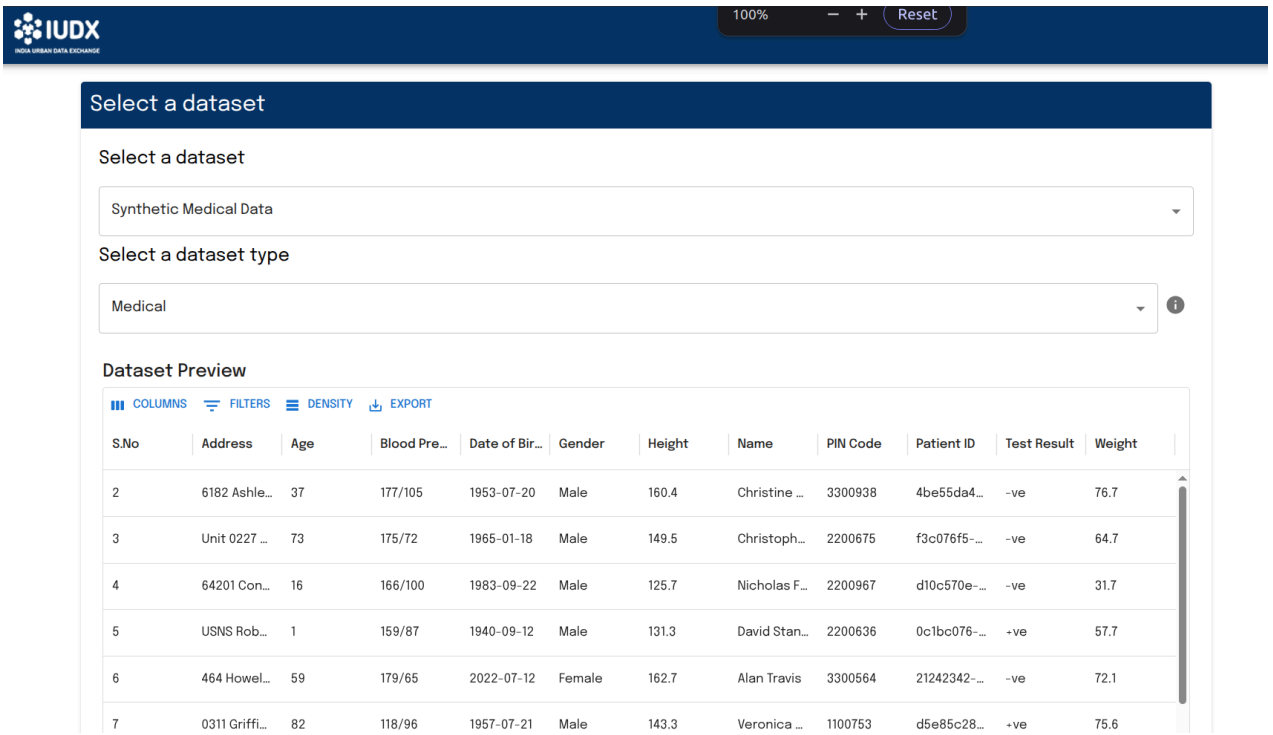
Fill in the required privacy parameters for Differential Privacy before proceeding.

Step 5a: Review the generated config file with your chosen parameters and confirm before selecting to run the algorithm in TEE or in normal mode.

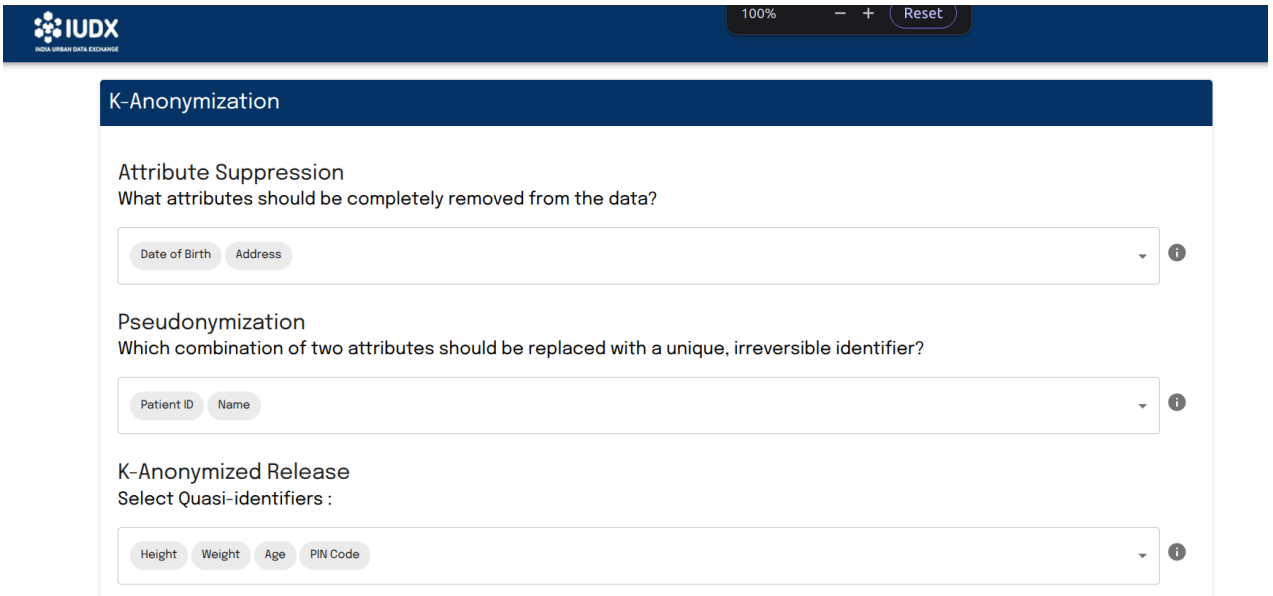
Step 6a: Fill in the job details including name, description, application name, dataset, and resource server URL before creating the job.



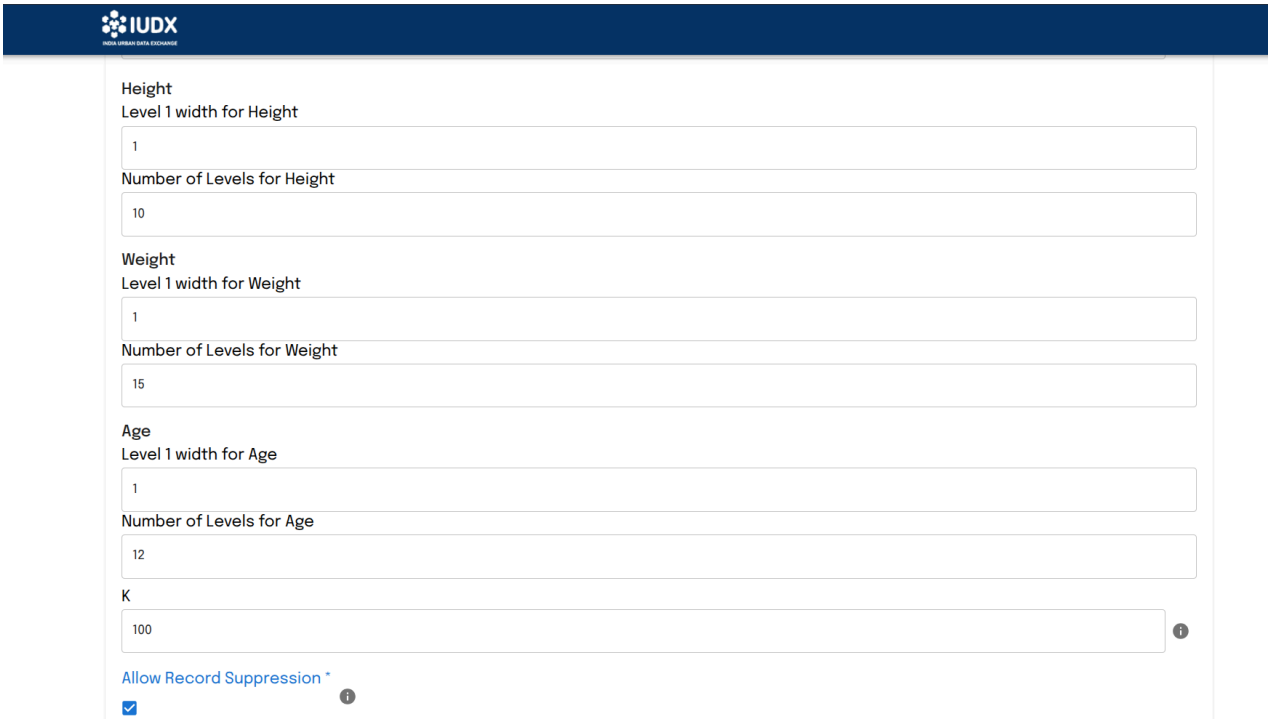
Step 7a: View the output as visualized data.



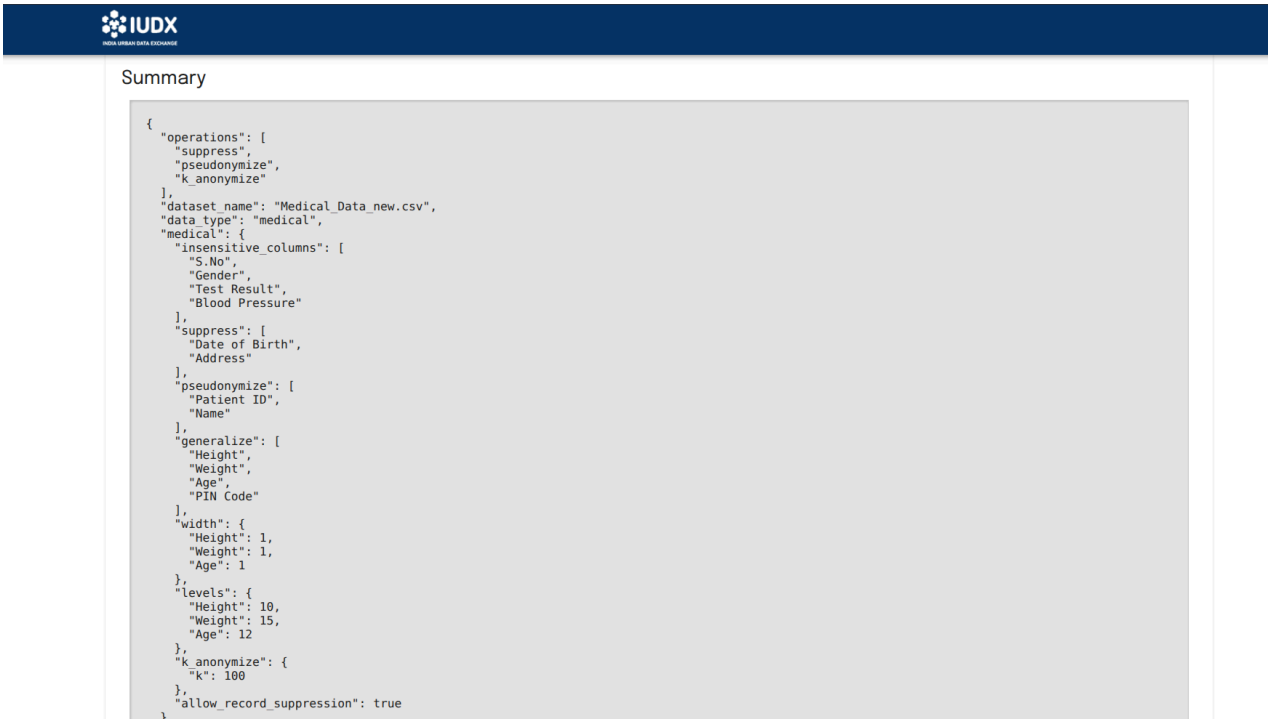
Step 3b: After clicking on K- anonymized Release, select the dataset and its type from the dropdown menus.



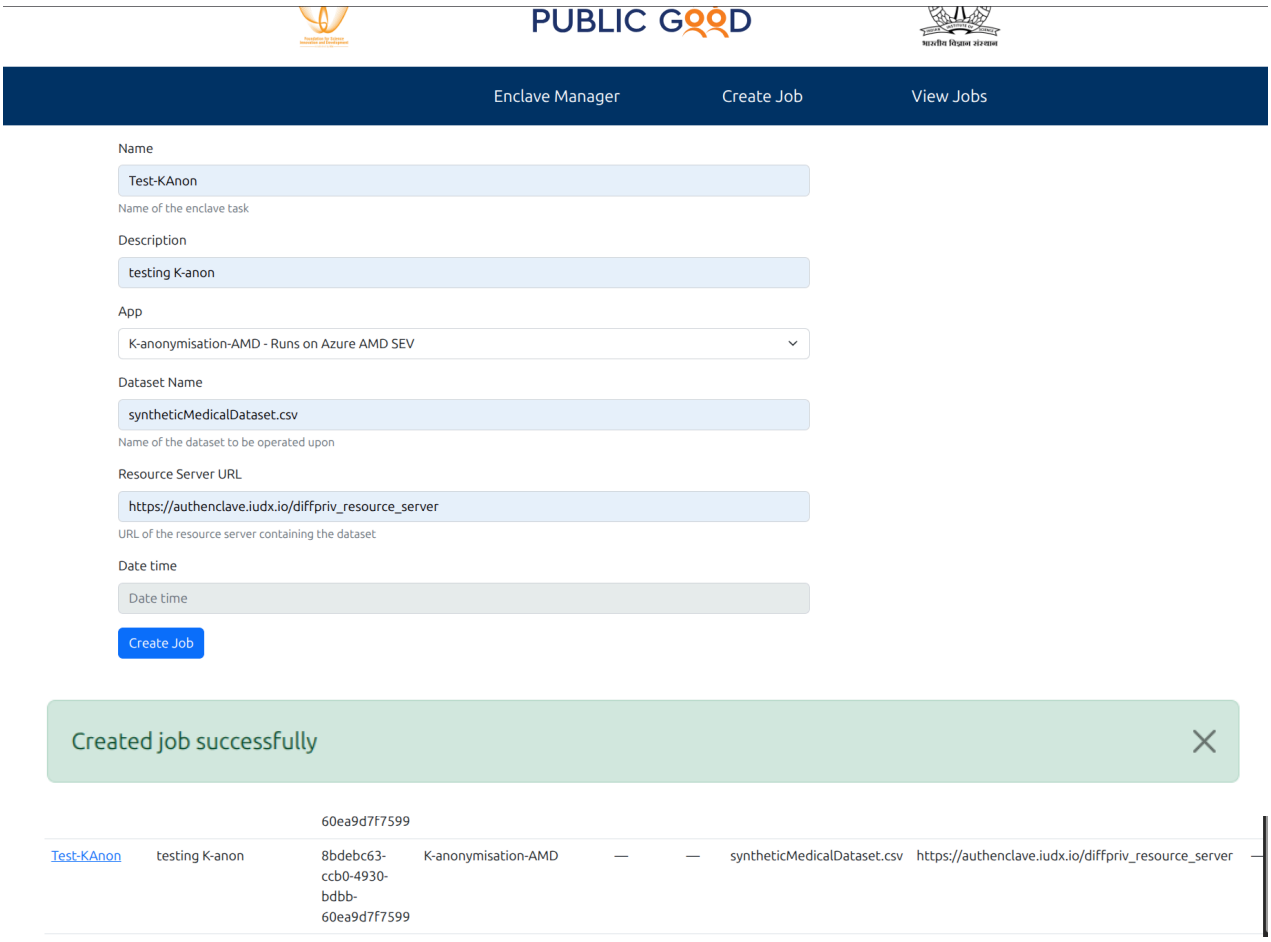
Step 4b: Select the parameters for k-anonymization by choosing columns to suppress, pseudonymize, and set as quasi-identifiers for generalization.



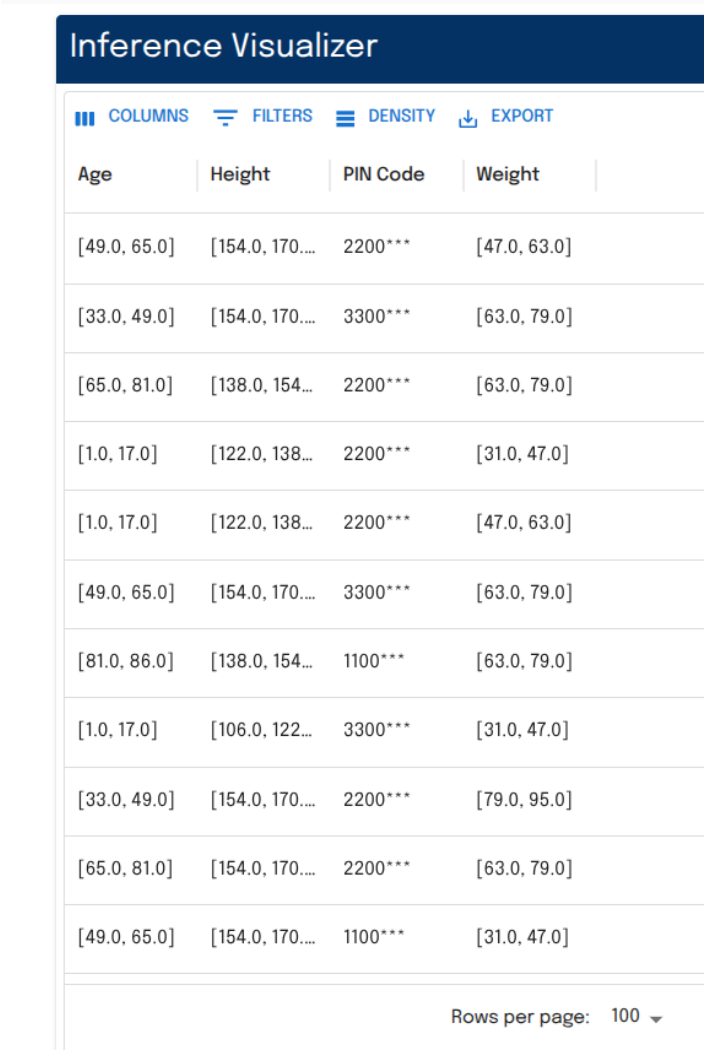
Choose the bin width and generalization levels for each selected quasi-identifier.



Step 5b: Review the generated config file with your chosen parameters and confirm before selecting to run the algorithm in TEE or in normal mode.



Step 6b: Fill in the job details including name, description, application name, dataset, and resource server URL before creating the job.



Step 7b: View the output containing your k-anonymized data.Second figure depicts only the equivalence classes from generalized output.

Appendix B: Github Repositories

Repository Name	Purpose	Main Files	Ports
iudx-dp-pipeline-ui-v2	UI for uploading datasets and selecting anonymization. Routes to pipelines.	App.tsx,App.py	5173 (UI), 5000 (API)
arx-anonymization	Performs k-anonymization via ARX API.	ARXTestingService.java, ARXTestingController.java	8070

differential-privacy	Applies differential privacy during anonymization.	iudx_dp_server.py, iudx_dp_main.py	6000
resource-server	Stores uploaded datasets and outputs	_init_.py	RS link